D2 — Attack Surface and Working Harness

1. Attack surface analysis (Track B)

Entry points in libheif (1.23.4, pinned commit `08075ae`) that parse untrusted input.

Most fuzzers go through `heif_context_read_from_memory` and the handle/decode calls that follow it. Two fuzzers (`box_fuzzer`, `color_conversion_fuzzer`) call internal functions directly instead of going through that API.

#	Entry point	Input it parses	Reachable from the memory-decode API? Guard or condition first	Harnessed by
1	ISOBMFF box parser (`ftyp`, `meta`, `moov`, …)	Box headers, sizes, nesting	Yes. First guard: input must pass the `ftyp` brand check (`heif_check_filetype`)	`file_fuzzer`, `api_fuzzer`
2	Single-box parser	One box with no container	No. Called directly on an internal box-parsing function, with no file-level guards	`box_fuzzer`
3	Item / property tables (`iinf`, `iloc`, `iref`, `ipco`/`ipma`)	Item IDs, extents, references, properties	Yes. Needs a well-formed `meta` box whose item tables agree with each other	`api_fuzzer`
4	Color profile (`colr`: nclx and ICC)	Profile bytes and matrix codes	Yes. Needs an image item with an associated `colr` property	`api_fuzzer`
5	EXIF / XMP metadata blocks	Raw metadata payloads	Yes. Needs an item linked to the image through a `cdsc` reference	`file_fuzzer`
6	Region items and masks	Region geometry and inline masks	Yes. Needs a region item referenced from an image	`api_fuzzer`
7	Grid / tile decode	Tile offsets, sizes, grid layout	Yes. Needs a `grid` item whose tiles resolve to valid items	`tile_fuzzer`
8	Codec bitstreams (HEVC: libde265; AV1: aom)	Compressed pixel data handed to third-party decoders	Yes, but only if the codec library is built in. The pinned Docker image includes them; a bare host without them never reaches this code	`file_fuzzer`, `api_fuzzer`
9	Uncompressed codec (`unci`, ISO 23001-17)	Component layout, tiling, deflate/zlib/brotli payloads	Yes. Off by default in libheif; our build script turns on the `WITH_UNCOMPRESSED_CODEC` option	`file_fuzzer`, `api_fuzzer`
10	Sequence / track decode (`moov`, `trak`, sample tables)	Track boxes and samples	Yes. Needs a valid sequence track with consistent sample tables	`sequence_fuzzer`, `api_fuzzer`
11	Color conversion	Pixel planes and conversion parameters taken from the fuzz input	No. The fuzzer builds images straight from the input bytes; no file parsing is involved	`color_conversion_fuzzer`
12	Encoder path	Encoding parameters and image planes	No. This is an encode API path, not decode. It needs an encoder library (x265/aom)	`encoder_fuzzer`

Outside our D1 decode scope: the encoder path (row 12), and entry points that come from the file I/O and custom-reader APIs (`heif_context_read_from_file`, `heif_context_read_from_reader`). Those wrap the same parser as row 1, so the memory API covers the parsing logic behind them.

2. Programs and test cases

Program	Purpose	Build status
file_fuzzer	Decode a HEIF/HEIC/AVIF file, including thumbnails and metadata	Builds
api_fuzzer	Exercise extended API surfaces (items, regions, depth, tracks)	Builds
box_fuzzer	Parse individual boxes	Builds

Program	Purpose	Build status
color_conversion_fuzzer	Color conversion paths	Builds
tile_fuzzer	Tile grid decoding	Builds
sequence_fuzzer	Image sequences	Builds
encoder_fuzzer	Encoding paths	Builds

No fuzzer failed to build. All are the upstream targets, unchanged. The seed corpus is the upstream `fuzzing/data/corpus` directory (133 files). Each run starts from a fresh copy of it.

3. Repository

https://github.com/lordzadd/libheif-fuzz-d2

4. Exact commands to run the harness

Recommended: a pinned environment. The Ubuntu 24.04 image is pinned by digest, apt packages come from a dated archive snapshot, and the image provides clang 18 plus the libde265/x265/aom codecs.

```
git clone https://github.com/lordzadd/libheif-fuzz-d2.git
cd libheif-fuzz-d2
docker build -t libheif-fuzz-d2 .
docker run --rm -v "$PWD/results:/repo/results" libheif-fuzz-d2
```

Native alternative (Linux/macOS with clang+libFuzzer, CMake ≥ 3.16, make, Python 3):

```
python3 harness/run.py        # 5 s per fuzzer (smoke test)
python3 harness/run.py 60     # longer runs
```

The harness prints a JSON summary with `total_crashes` and writes `results/fuzz_results.csv`. The CSV has one row per fuzzer: exit code, elapsed time, crashes, timeouts/OOMs, executions, and final edge coverage. The default of 5 s per fuzzer is only a smoke test; real campaigns pass a longer duration.

AI Use Appendix

- **opencode**: drafted the original harness (`harness/run.py`), the build script, and the first version of this document. I reviewed and edited the output, then tested a clean clone on macOS and on Ubuntu 24.04.
- **Claude Code (Claude Opus 5.5)**: reviewed the repo against the D2 requirements and ran a clean-clone test in a fresh Ubuntu container. It then made these changes: added the pinned `Dockerfile` and `.gitattributes`; enabled the uncompressed codec and built-in codecs in `build.sh`; fixed the harness's coverage parsing (the old `edges` column was always 0); reset the corpus and crash directories on every run; applied the upstream LSan suppressions; added a duration argument; and corrected the attack-surface table (wrong rows for `box_fuzzer` and `color_conversion_fuzzer`, plus entry points that were missing). I reviewed these changes.